

Meeting Real-Time Control Budgets for Vision-Language-Action Manipulation Policies on the NVIDIA Jetson Orin Nano

Project Abstract

by

CMPE 295A

Lam Nguyen (phuongduylam.nguyen@sjsu.edu)

Vi Thi Tuong Nguyen (thituongvi.nguyen@sjsu.edu)

James Pham (james.pham03@sjsu.edu)

Project Advisor

Dr. Kaikai Liu

September 2026

ABSTRACT

Meeting Real-Time Control Budgets for Vision-Language-Action Manipulation Policies on the NVIDIA Jetson Orin Nano

By

Lam Nguyen, Vi Thi Tuong Nguyen, James Pham

Vision-Language-Action (VLA) models combine visual perception with language-model-style reasoning in a single network, replacing the hand-engineered perception and planning pipelines used in conventional manipulation systems. Instead of separate modules that must be retuned for every task, a VLA model observes a scene, interprets an instruction, and directly outputs robot commands, which lets it generalize to unfamiliar objects, environments, and tasks. This has made VLA architectures a leading research direction for general-purpose, instruction-following robotic arms. Deploying these policies onboard the robot, rather than on a tethered workstation, is the remaining barrier to using them outside the lab.

All published benchmarks for Hy-Embodied-0.5-VLA run on datacenter GPUs. The model pairs a 4B-parameter backbone with a 370M-parameter flow-matching action expert, and its FP16 weights require roughly 8.7 GB. Our Jetson Orin Nano has 8 GB of unified memory, shared across CPU, GPU, operating system, and camera pipeline. Published edge deployments use larger boards, and we found no published results at this memory tier. Beyond the memory bottleneck, latency constrains replanning more than motion. Asynchronous action chunking keeps the arm moving on schedule, but each new chunk is computed from observations that are one inference cycle old.

We will start with a closed-loop control rate budget for a tabletop manipulation task, then find the smallest configuration of Hy-Embodied-0.5-VLA that runs on a Jetson Orin Nano while holding success rate within five percentage points of the full-precision baseline. We will experiment with combinations of post-training quantization, TensorRT compilation, flow-matching denoising steps, and action chunk size. Each configuration will be evaluated in RoboTwin 2.0 simulation and on a physical arm, measuring latency, control rate, memory, and power. We will then compare the best configurations against the full model running on a workstation GPU and against SmolVLA. The deliverable is an optimized Hy-Embodied-0.5-VLA configuration that runs the policy onboard the robot.